

Postman is best understood as an **API client + API testing and automation tool**. You can use it to send different HTTP requests with different methods, parameters, headers, authentication mechanisms, request bodies, variables, and test data.

The basic workflow is:

API → Request → Parameters/Headers/Auth/Body → Send → Response → Tests → Collection → Runner → Automation

Postman currently supports organizing requests into **collections**, using **environments and variables**, running collections through the **Collection Runner**, and automating runs through tools such as Postman CLI. [Postman Docs](#)

1. What is an API?

API — Application Programming Interface — is an interface through which one software application communicates with another.

For example:

```
Java Spring Boot Application
    |
    | HTTP Request
    ↓
  REST API
    |
    ↓
  Database
    |
    ↓
HTTP Response
```

Example:

GET <https://api.example.com/users/101>

The client asks:

Give me the user whose ID is 101.

The server might return:

```
{
  "id": 101,
  "name": "Anurag",
  "email": "anurag@example.com"
}
```

2. What is Postman?

Postman allows you to construct and execute API requests without having to write a separate program for every request.

You can specify:

Component	Example
HTTP Method	GET
URL	https://api.example.com/users/101
Path Parameter	101
Query Parameter	?page=2
Header	Authorization: Bearer token
Request Body	JSON/XML/form-data
Authentication	Bearer Token
Variables	{{base_url}}
Tests	JavaScript assertions
Collection	Group of requests
Environment	DEV/TEST/PROD

Postman collections can contain requests along with authorization, parameters, headers, bodies, tests and other settings. [Postman Docs](#)

3. Install and open Postman

You can use the Postman desktop application or web application. Postman recommends the desktop application for the best experience when sending API requests. [Postman Docs](#)

Basic workflow:

Open Postman



Create Workspace
↓
Create Collection
↓
Create Request
↓
Enter URL
↓
Select HTTP Method
↓
Configure parameters
↓
Configure headers
↓
Configure authentication
↓
Configure body
↓
Send
↓
Inspect response

4. HTTP methods in Postman

The most important methods are:

Method	Purpose	Typical operation
GET	Retrieve data	Read user
POST	Create data	Create user
PUT	Replace/update resource	Replace user
PATCH	Partially update resource	Change email
DELETE	Delete resource	Delete user
HEAD	Retrieve headers	Check resource
OPTIONS	Discover supported operations	CORS/API capabilities

For REST APIs, GET, POST, PUT, PATCH and DELETE are the methods you'll use most frequently.
[Postman Docs](#)

5. GET API

Suppose the API is:

GET <https://api.example.com/users>

In Postman:

Method:

GET

URL:

<https://api.example.com/users>

Click:

Send

Possible response:

```
{
  "users": [
    {
      "id": 1,
      "name": "Anurag"
    },
    {
      "id": 2,
      "name": "Rahul"
    }
  ]
}
```

When to use GET

Use GET when you want to **retrieve information**.

Examples:

GET /users

GET /users/101

GET /products

GET /orders/5001

6. GET with Query Parameters

Suppose the API supports:

GET /users?page=2&size=10

In Postman, you can enter:

https://api.example.com/users

Then select:

Params

KEY	VALUE
page	2
size	10

Postman constructs:

https://api.example.com/users?page=2&size=10

Another example

GET /products?category=mobile&minPrice=10000&maxPrice=50000

Parameters:

KEY	VALUE
category	mobile
minPrice	10000
maxPrice	50000

7. Path Parameters

A path parameter is part of the URL path.

Example:

GET /users/{id}

Actual request:

GET /users/101

Another:

GET /users/205

Another:

GET /users/500

You can make it dynamic:

GET {{base_url}}/users/{{user_id}}

Environment:

base_url = https://api.example.com

user_id = 101

Postman sends:

https://api.example.com/users/101

Variables can be referenced using double curly braces such as {{base_url}}. [Postman Docs](#)

8. Query Parameter vs Path Parameter

Feature	Path Parameter	Query Parameter
Location	URL path	After ?
Example	/users/101	/users?page=2
Usually identifies resource	Yes	Usually filters/configures request
Required	Often	Often optional
Example	userId=101	page=2
Postman	URL/path	Params tab

Example

GET /users/101/orders?page=2&size=10

Here:

101

is a path parameter.

page=2
size=10

are query parameters.

9. POST API

POST is commonly used to create a new resource.

Example:

POST https://api.example.com/users

Go to:

Body
→ raw
→ JSON

Enter:

```
{  
  "name": "Anurag",  
  "email": "anurag@example.com",  
  "age": 30  
}
```

Header:

Content-Type: application/json

Click:

Send

Possible response:

```
{
  "id": 101,
  "name": "Anurag",
  "email": "anurag@example.com",
  "age": 30
}
```

10. PUT API

PUT is generally used to replace or update a resource.

PUT <https://api.example.com/users/101>

Body:

```
{
  "name": "Anurag Bharti",
  "email": "anurag.bharti@example.com",
  "age": 31
}
```

Conceptually:

Existing Resource

↓

PUT

↓

Replacement representation

11. PATCH API

PATCH is generally used for a partial update.

Example:

PATCH <https://api.example.com/users/101>

Body:

```
{
  "email": "newemail@example.com"
}
```

Instead of sending the complete user object, you're changing only the email.

12. PUT vs PATCH

PUT	PATCH
Usually complete replacement	Partial modification
Usually sends complete representation	Sends changed fields
/users/101	/users/101
Larger request body	Usually smaller body
Idempotent by HTTP semantics	May be idempotent depending on operation

13. DELETE API

Example:

DELETE https://api.example.com/users/101

Usually no request body is necessary.

Click:

Send

Possible response:

204 No Content

or:

```
{  
  "message": "User deleted successfully"  
}
```

14. Headers

HTTP headers provide additional information about the request.

Example:

Content-Type: application/json

Accept: application/json

Authorization: Bearer <token>

In Postman:

Headers

KEY	VALUE
Content-Type	application/json
Accept	application/json
Authorization	Bearer eyJ...

Common headers:

Header	Purpose
Content-Type	Type of request body
Accept	Desired response format
Authorization	Authentication credentials
User-Agent	Identifies client
Cache-Control	Cache behavior
X-API-Key	API key, when an API uses one

15. Authentication

Many APIs require authentication.

In Postman:

Authorization

You may encounter:

Authentication	Example
No Auth	Public API
API Key	X-API-Key: abc123
Bearer Token	Authorization: Bearer token

Basic Auth	Username + password
OAuth 2.0	Access token
JWT	Bearer JWT
AWS Signature	AWS APIs

16. Basic Authentication

Suppose:

Username = admin

Password = password123

Postman:

Authorization

→ Type: Basic Auth

→ Username: admin

→ Password: password123

Postman generates the appropriate authorization header.

17. Bearer Token

Suppose you receive:

eyJhbGciOiJIUzI1NiIs...

Postman:

Authorization

→ Type: Bearer Token

→ Token: eyJhbGciOiJIUzI1NiIs...

Request becomes conceptually:

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

18. API Key

Example:

GET <https://api.example.com/weather>

API key:

abc123xyz

Depending on the API specification, it could be sent as a header:

X-API-Key: abc123xyz

or as a query parameter:

?api_key=abc123xyz

Always follow the API's documentation because the expected location/name of an API key varies.

19. Request Body types in Postman

For POST/PUT/PATCH, Postman supports different body formats.

Body type	Typical use
none	No body
form-data	File upload + fields
x-www-form-urlencoded	HTML-style form
raw JSON	REST APIs
raw XML	XML APIs
binary	Binary file
GraphQL	GraphQL requests

20. JSON Request

Example:

```
{  
  "firstName": "Anurag",  
  "lastName": "Bharti",  
  "email": "anurag@example.com"  
}
```

Set:

Body
→ raw
→ JSON

21. Form-data

Suppose an API uploads a profile picture.

Body
→ form-data

KEY	TYPE	VALUE
name	Text	Anurag
email	Text	<u>anurag@example.com</u>
profileImage	File	photo.jpg

This is commonly used when an API needs both fields and uploaded files.

22. x-www-form-urlencoded

Example:

username=anurag
password=secret

Postman:

Body

→ x-www-form-urlencoded

KEY	VALUE
username	anurag
password	secret

23. Variables — one of Postman's most important concepts

Instead of repeatedly writing:

https://api.example.com

create:

base_url = https://api.example.com

Then:

{{base_url}}/users

Postman replaces:

{{base_url}}

with its configured value when sending the request. [Postman Docs](#)

24. Types of variables

Important Postman variable scopes include:

Variable	Scope
Global	Workspace-wide
Collection	Collection-wide
Environment	Current environment
Data	Collection Runner input
Local	Current request/run context

Postman documentation notes that collection variables apply throughout a collection, environment variables allow different values for contexts such as local/testing/production, and data variables can come from CSV/JSON files during collection runs. [Postman Docs](#)

25. DEV / TEST / PROD example

This is one of the most useful real-world applications.

DEV

```
base_url =  
http://localhost:8080
```

TEST

```
base_url =  
https://test-api.company.com
```

PROD

```
base_url =  
https://api.company.com
```

Request remains:

```
GET {{base_url}}/users
```

You don't have to create three different requests.

Simply change the active environment.

Postman environments are specifically designed to let the same request work against different servers such as local, test, and production. [Postman Docs](#)

26. Practical environment example

Create:

DEV

Variables:

Variable	Value
base_url	<u>http://localhost:8080</u>
username	devuser
token	dev-token

Create:

TEST

Variables:

Variable	Value
base_url	<u>https://test.example.com</u>
username	testuser
token	test-token

Create:

PROD

Variables:

Variable	Value
base_url	<u>https://api.example.com</u>
username	produser
token	prod-token

Then use:

{{base_url}}/users

and:

Authorization: Bearer {{token}}

Changing the environment changes the actual values used by the request. [Postman Docs](#)

27. Dynamic variables

Postman also provides dynamic values.

For example:

```
{{ $randomFirstName }}
```

or other supported dynamic variables.

This is useful when you don't want to manually type unique test data.

Postman also supports dynamic variables through scripting with `pm.variables.replaceIn()`. [Postman Docs](#)

28. Running the same API with different inputs

Suppose:

```
POST {{base_url}}/users
```

You want to test:

Input 1

```
{  
  "name": "Anurag",  
  "age": 30  
}
```

Input 2

```
{  
  "name": "Rahul",  
  "age": 25  
}
```

Input 3

```
{  
  "name": "Priya",  
  "age": 28  
}
```

You can manually change the body and click:

Send

But this becomes inefficient when you have hundreds of inputs.

For that, use:

Collection Runner + CSV/JSON data.

29. Data-driven testing with CSV

Create:

users.csv

Example:

```
name,email,age
Anurag,anurag@example.com,30
Rahul,rahul@example.com,25
Priya,priya@example.com,28
Amit,amit@example.com,35
```

Request body:

```
{
  "name": "{{name}}",
  "email": "{{email}}",
  "age": {{age}}
}
```

Run the collection using the CSV file.

Conceptually:

Iteration 1
Anurag

Iteration 2
Rahul

Iteration 3
Priya

Iteration 4

Amit

Postman's Collection Runner supports CSV and JSON data files for repeated data-driven execution.
[Postman Docs](#)

30. Data-driven testing with JSON

Example:

```
[
  {
    "name": "Anurag",
    "email": "anurag@example.com",
    "age": 30
  },
  {
    "name": "Rahul",
    "email": "rahul@example.com",
    "age": 25
  }
]
```

Request:

```
{
  "name": "{{name}}",
  "email": "{{email}}",
  "age": {{age}}
}
```

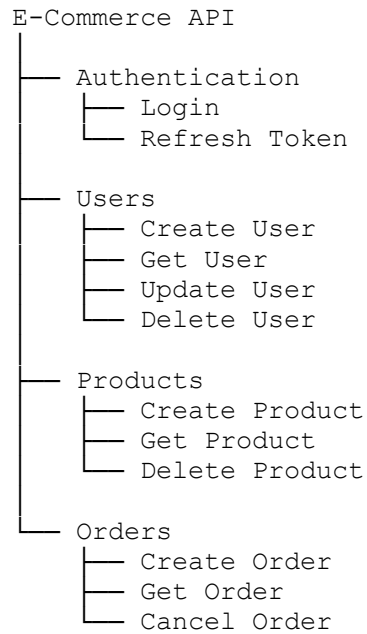
Runner:

```
JSON file
↓
Iteration 1
↓
POST API
↓
Iteration 2
↓
POST API
```

31. Collection

A **collection** is a logical group of API requests.

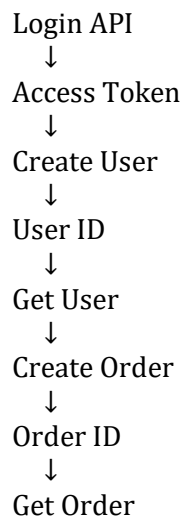
For example:



Collections allow you to organize related requests into folders and subfolders. [Postman Docs](#)

32. Chaining APIs

A very important real-world scenario is:



Suppose Login returns:

```
{
  "accessToken": "abc123",
  "userId": 101
}
```

You can save:

```
accessToken
userId
```

as variables and use them in subsequent requests.

For example:

```
Authorization:
Bearer {{accessToken}}
```

and:

```
GET {{base_url}}/users/{{userId}}
```

This is called **request chaining**.

Postman supports using scripts and variables to pass values between requests. [Postman Blog](#)

33. Pre-request Script

A **pre-request script** runs before the API request is sent.

Example:

```
pm.environment.set("userId", "101");
```

Then the request can use:

```
{{userId}}
```

Another example:

```
const timestamp = Date.now();
pm.environment.set("timestamp", timestamp);
```

Request:

GET {{base_url}}/users/{{userId}}?timestamp={{timestamp}}

34. Post-response script / Test

After receiving the response, you can validate it.

Example:

```
pm.test("Status code is 200", function () {  
  pm.response.to.have.status(200);  
});
```

Another:

```
pm.test("Response contains user id", function () {  
  const jsonData = pm.response.json();  
  
  pm.expect(jsonData.id).to.exist;  
});
```

35. Validate JSON response

Suppose:

```
{  
  "id": 101,  
  "name": "Anurag",  
  "email": "anurag@example.com"  
}
```

Test:

```
const response = pm.response.json();  
  
pm.test("User ID is 101", function () {  
  pm.expect(response.id).to.eql(101);  
});  
  
pm.test("Name is Anurag", function () {  
  pm.expect(response.name).to.eql("Anurag");  
});
```

36. Validate response time

```
pm.test("Response time is less than 1000 ms", function () {  
  pm.expect(pm.response.responseTime).to.be.below(1000);  
});
```

```
});
```

This allows you to introduce basic performance expectations into functional API tests.

37. Extract token from Login API

Suppose Login response:

```
{
  "token": "abc123xyz"
}
```

Post-response script:

```
const response = pm.response.json();

pm.environment.set("token", response.token);
```

Next API:

GET {{base_url}}/users

Authorization:

Bearer {{token}}

Flow:

```
LOGIN
↓
response.token
↓
environment variable
↓
{{token}}
↓
GET /users
```

38. Extract ID and pass it to another API

Response:

```
{
  "id": 501,
  "name": "Anurag"
```

```
}
```

Script:

```
const response = pm.response.json();  
pm.environment.set("userId", response.id);
```

Next request:

```
GET {{base_url}}/users/{{userId}}
```

Result:

```
GET https://api.example.com/users/501
```

39. Complete real-world API workflow

Consider an e-commerce application.

Step 1 — Login

```
POST /auth/login
```

Body:

```
{  
  "username": "anurag",  
  "password": "password"  
}
```

Response:

```
{  
  "token": "abc123"  
}
```

Save:

```
{{token}}
```

Step 2 — Get products

```
GET /products
```

Header:

Authorization: Bearer {{token}}

Step 3 — Create order

POST /orders

Body:

```
{  
  "productId": 101,  
  "quantity": 2  
}
```

Response:

```
{  
  "orderId": 5001  
}
```

Save:

{{orderId}}

Step 4 — Get order

GET /orders/{{orderId}}

Step 5 — Cancel order

DELETE /orders/{{orderId}}

Complete workflow:

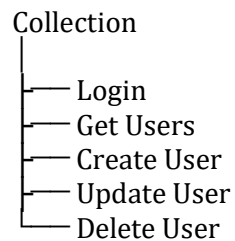
```
Login  
↓  
Token  
↓  
Get Products  
↓  
Create Order  
↓  
Order ID  
↓  
Get Order  
↓  
Cancel Order
```

This is much closer to how Postman is used in professional API testing than simply sending isolated GET requests.

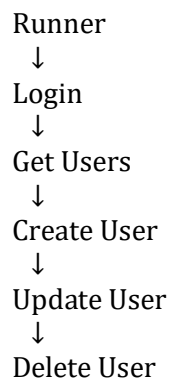
40. Collection Runner

The **Collection Runner** allows you to execute some or all requests in a collection in a specified order. It can also use scripts and data files during the run. [Postman Docs](#)

Example:



Run:



You can configure the environment, iterations, data files and other run settings.

41. Different inputs + Collection Runner

Suppose:

POST /users

Data:

name,email
Anurag,anurag@example.com
Rahul,rahul@example.com

Priya,priya@example.com

Request:

```
{  
  "name": "{{name}}",  
  "email": "{{email}}"  
}
```

Runner:

Iteration 1

↓

Anurag

Iteration 2

↓

Rahul

Iteration 3

↓

Priya

This is particularly useful for **data-driven API testing**.

42. Multiple API environments

A professional Postman project might have:

DEV

TEST

UAT

STAGING

PROD

Each environment can have:

base_url

client_id

client_secret

token

user_id

For example:

Environment	base_url
DEV	http://localhost:8080
TEST	https://test-api.company.com
UAT	https://uat-api.company.com
PROD	https://api.company.com

Same request:

GET {{base_url}}/users

Different environment:

DEV → localhost

TEST → test server

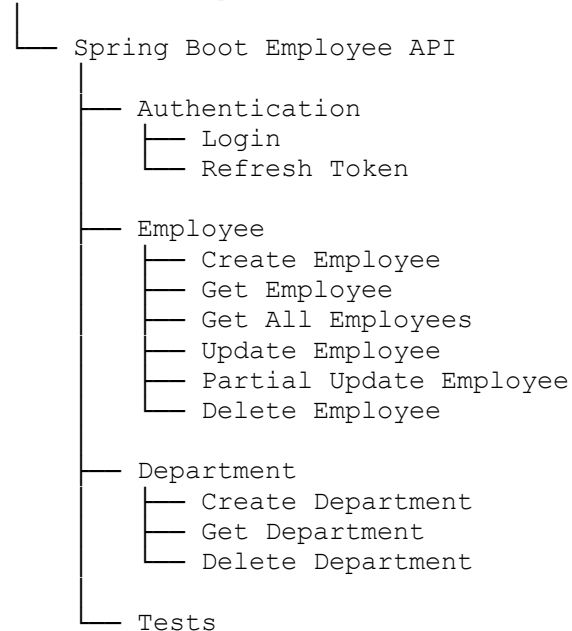
UAT → UAT server

PROD → production server

43. Recommended Postman project structure

For a Spring Boot REST API, I'd recommend:

Postman Workspace



Environment:

SpringBoot-DEV

Variables:

base_url

token

employee_id

department_id

44. Complete request template

A professional request can be thought of as:

HTTP METHOD

+

URL

+

PATH PARAMETERS

+

QUERY PARAMETERS

+

HEADERS

+

AUTHENTICATION

+

REQUEST BODY

↓

SEND

↓

STATUS CODE

+

RESPONSE HEADERS

+

RESPONSE BODY

+

RESPONSE TIME

↓

TESTS

For example:

POST

{{base_url}}/employees/{{employee_id}}?notify=true

Authorization:

Bearer {{token}}

Content-Type:
application/json

Body:

```
{  
  "name": "{{employee_name}}",  
  "department": "{{department}}"  
}
```

45. Common mistakes in Postman

Problem	Possible reason
400 Bad Request	Invalid request/body
401 Unauthorized	Missing/invalid authentication
403 Forbidden	Insufficient permission
404 Not Found	Wrong URL/resource
405 Method Not Allowed	Wrong HTTP method
409 Conflict	Resource conflict
415 Unsupported Media Type	Incorrect Content-Type
422 Unprocessable Entity	Validation/business-rule failure
429 Too Many Requests	Rate limit
500 Internal Server Error	Server-side problem

46. Best practice for testing APIs

Use this order:

1. Understand API documentation
2. Identify endpoint
3. Identify HTTP method
4. Identify path parameters
5. Identify query parameters
6. Identify headers

7. Identify authentication
 8. Identify request body
 9. Send request
 10. Check status code
 11. Check response headers
 12. Check response body
 13. Write tests
 14. Extract dynamic values
 15. Chain requests
 16. Create collection
 17. Create environments
 18. Run with different data
 19. Automate
-

47. Postman tutorial/video learning path

Rather than watching random videos, I recommend following this sequence.

Level 1 — Beginner

1. What is Postman?

Learn:

- API
- REST API
- HTTP
- request
- response
- endpoint
- status code

2. Postman UI

Learn:

- Workspace
- Collections
- Requests
- History
- Console
- Environment selector

3. HTTP methods

Learn:

GET

POST
PUT
PATCH
DELETE

4. Parameters

Learn:

Path parameters
Query parameters
Headers
Request body

Level 2 — API request handling

5. JSON

Learn:

```
{  
  "name": "Anurag",  
  "age": 30  
}
```

6. Authentication

Learn:

Basic Auth
API Key
Bearer Token
JWT
OAuth 2.0

7. Collections

Learn:

Collection
Folder
Request
Request ordering

Level 3 — Variables

8. Variables

Learn:

Global
Collection
Environment
Data
Local

9. Environments

Learn:

DEV
TEST
UAT
PROD

10. Dynamic variables

Learn:

Random values
Dynamic test data

Postman's official documentation has dedicated material for variables and environments. [Postman Docs](#)

Level 4 — Postman scripting

11. Pre-request scripts

Learn:

`pm.environment.set(...)`

12. Post-response scripts

Learn:

`pm.test(...)`

13. Response parsing

Learn:

`pm.response.json()`

14. Request chaining

Learn:

Login

↓

Token

↓

User API

↓

ID

↓

Order API

Level 5 — Automation

15. Collection Runner

Learn:

- Run collection
- Run folder
- Iterations
- Environment
- Data file
- Execution order

16. CSV data

`users.csv`

17. JSON data

`users.json`

18. Automated API testing

Learn:

Assertions

Status-code validation

Response validation

Schema validation

Postman's current documentation specifically covers Collection Runner execution, test results, data-driven runs, and automation options. [Postman Docs](#)

48. Recommended Postman video tutorials

1. Postman official — Running Collections



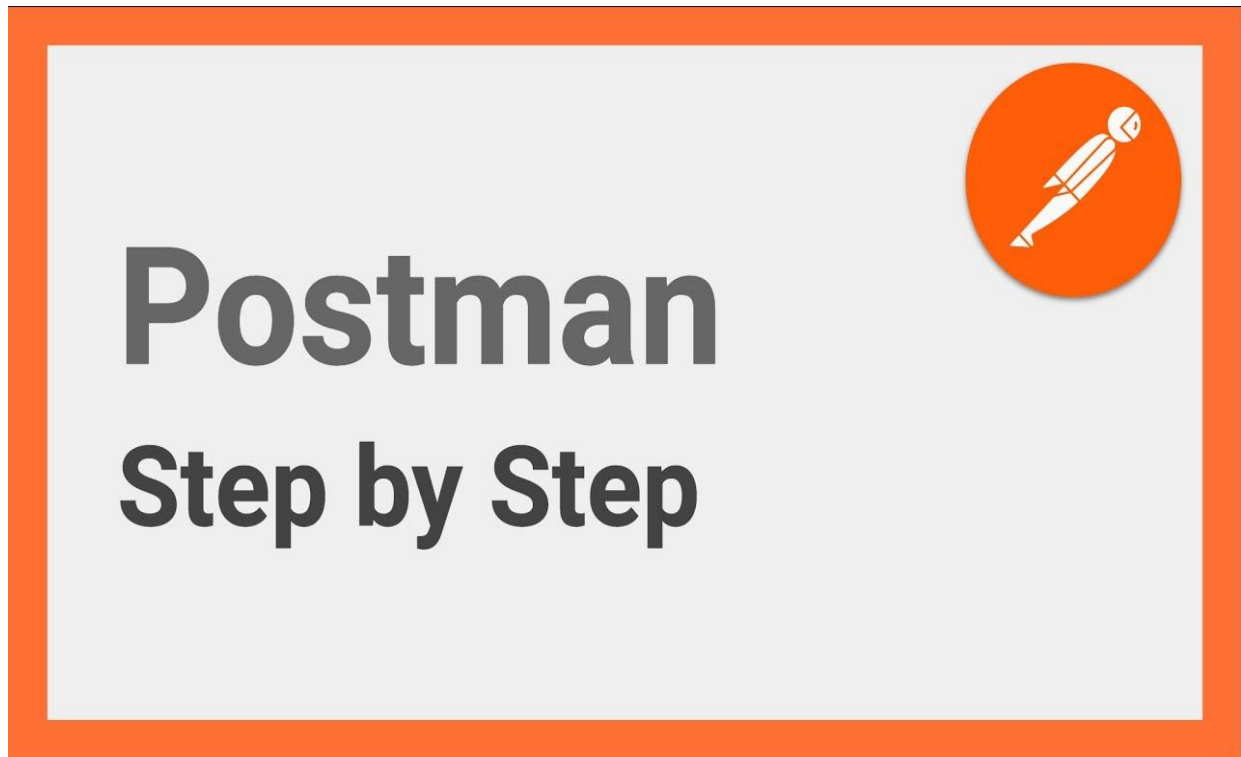
[Running Collections in Postman: Automate Your API Testing youtube.com](#)

This is useful for learning:

- Collections
- Collection Runner
- Tests
- Multiple requests
- Environments
- CSV/JSON data
- Request chaining
- Postman Console

It is an official Postman tutorial, although it is an older video, so use it primarily for the concepts and pair it with the current documentation. [YouTube](#)

2. Automation Step by Step — Postman API Testing



[Learn Postman for API Testing — Step-by-Step Tutorial
youtube.com](https://www.youtube.com/watch?v=...)

This is a particularly good **beginner-to-intermediate** tutorial because it covers:

Postman basics



GET



POST



PUT



PATCH



DELETE



Collections



Variables



Scripts



Environments

↓
Tests

The tutorial was published in September 2024 and includes timestamps for the major topics.
[YouTube](#)

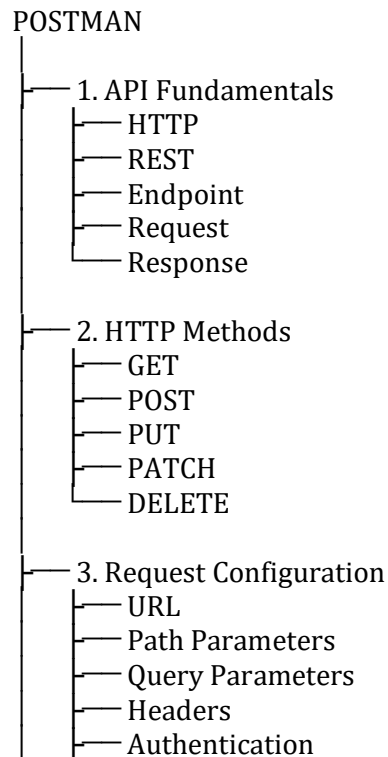
49. Official Postman documentation/tutorial sequence

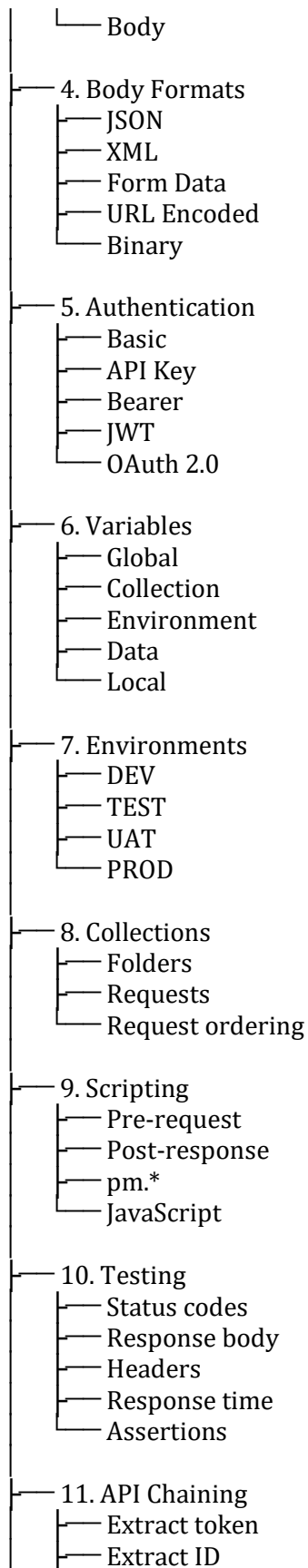
For current Postman functionality, I recommend using the official learning material alongside videos:

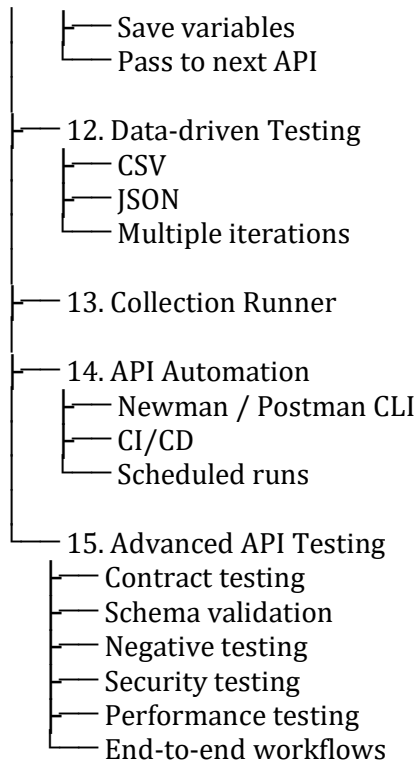
1. [Postman Basics](#)
 2. [Postman Elements — Collections, Requests, Environments, etc.](#)
 3. [Variables and Environments](#)
 4. [Environment Variables](#)
 5. [Collection Runner](#)
 6. [Postman API Testing Documentation](#)
-

50. The complete Postman learning roadmap

For your **Java + Spring + Spring Boot REST API** learning, I recommend mastering Postman in this exact order:







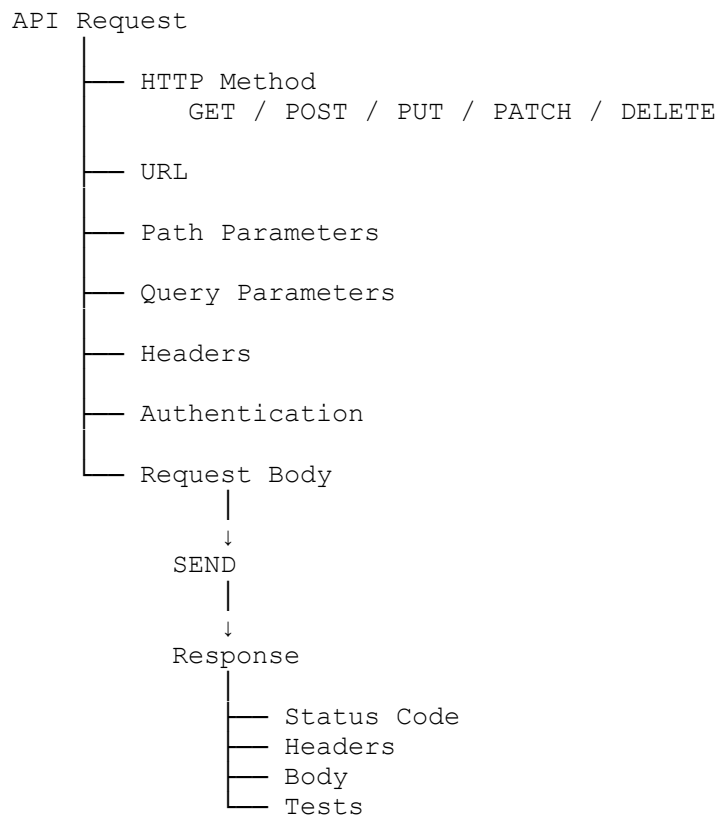
The key idea to remember: don't create a separate Postman request for every possible input. Create a reusable request using **variables**, select the appropriate **environment**, and use **CSV/JSON data + Collection Runner** when you need to execute the same API with many different inputs. That is the foundation for moving from basic Postman usage to professional API automation. [Postman Docs](#)

Yes. The most useful way to learn Postman is to understand **how to create one reusable API request and then run it with different URLs, parameters, headers, authentication values, and input data.**

Postman lets you build HTTP requests with parameters, headers and body data, save them in collections, and reuse them for testing and automation. [Postman Docs](#)

1. Basic Postman concept

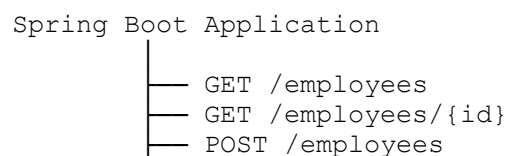
Think of a Postman API request as:



2. What is an API?

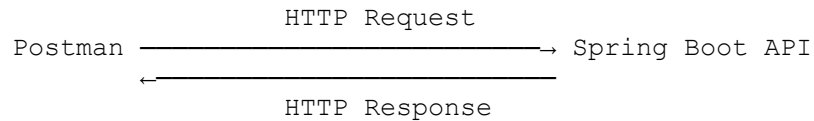
An **API — Application Programming Interface** — is a mechanism that allows one software application to communicate with another.

For example, suppose you have a Spring Boot application:



- PUT /employees/{id}
- PATCH /employees/{id}
- DELETE /employees/{id}

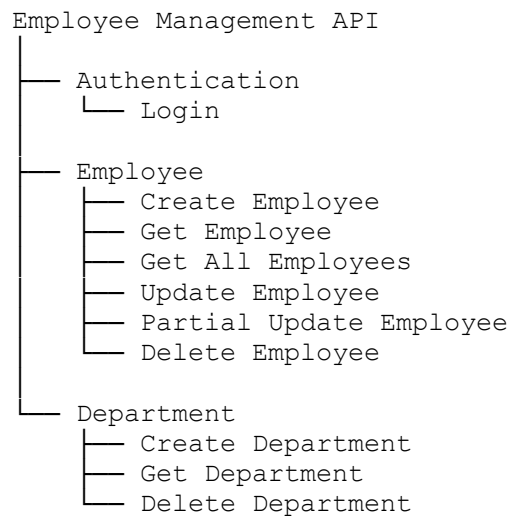
Postman acts as the client:



3. Create your first Postman collection

A **Collection** is a group of related API requests.

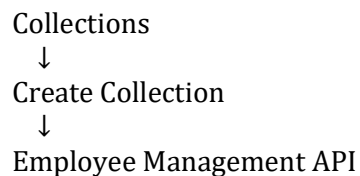
For example:



Postman collections are intended for saving, organizing, sharing, testing, documenting and automating requests. [Postman Docs](#)

Create it

In Postman:



Then create folders:

Authentication
Employee
Department

4. Create a GET API

Suppose your Spring Boot API is running at:

`http://localhost:8080`

Create:

GET
`http://localhost:8080/employees`

Click:

Send

Possible response:

```
[
  {
    "id": 1,
    "name": "Anurag",
    "department": "IT"
  },
  {
    "id": 2,
    "name": "Rahul",
    "department": "HR"
  }
]
```

The basic steps are:

New Request
↓
Select GET
↓
Enter URL
↓
Send
↓
Read Response

Postman displays the returned response so you can inspect and troubleshoot it. [Postman Docs](#)

5. GET API with a path parameter

Suppose your API is:

GET /employees/{id}

In Postman you can write:

GET http://localhost:8080/employees/:id

Postman supports path parameters using a : placeholder. [Postman Docs](#)

For example:

:id = 101

The actual request becomes:

GET http://localhost:8080/employees/101

Response:

```
{
  "id": 101,
  "name": "Anurag",
  "department": "IT"
}
```

Change the parameter:

:id = 102

and you send:

GET http://localhost:8080/employees/102

Important concept

/employees/:id
↑
Path Parameter

The path parameter normally identifies a particular resource.

6. GET API with query parameters

Suppose you want:

GET /employees?department=IT&city=Mumbai

In Postman:

GET
http://localhost:8080/employees

Open:

Params

Enter:

KEY	VALUE
department	IT
city	Mumbai

Postman generates:

http://localhost:8080/employees?department=IT&city=Mumbai

Query parameters occur after ? and multiple parameters are separated with &. Postman lets you enter them directly in the URL or through the **Params** tab. [Postman Docs](#)

7. Path parameter vs query parameter

This distinction is extremely important.

Feature	Path Parameter	Query Parameter
Example	/employees/101	/employees?department=IT
Syntax	:id in Postman	Params tab
Purpose	Identify a resource	Filter/configure a request
Example	Employee ID	Department
Typical API	/employees/{id}	/employees?department=IT

Example

GET /employees/101?includeSalary=true

Here:

101

is a path parameter.

includeSalary=true

is a query parameter.

8. Create a POST API

Suppose your Spring Boot API is:

POST /employees

Postman:

POST

http://localhost:8080/employees

Select:

Body

↓

raw

↓

JSON

Enter:

```
{
  "name": "Anurag",
  "email": "anurag@example.com",
  "department": "IT",
  "salary": 80000
}
```

Click:

Send

Possible response:

```
{
  "id": 101,
  "name": "Anurag",
  "email": "anurag@example.com",
  "department": "IT",
  "salary": 80000
}
```

Postman supports several request-body formats including form-data, URL-encoded, raw, binary and GraphQL; JSON is commonly sent using raw body mode. [Postman Docs](#)

9. Create different POST requests with different inputs

You could create separate requests:

Create Employee - Anurag
Create Employee - Rahul
Create Employee - Priya

But **don't do this when the API structure is the same.**

Instead create one reusable request:

POST {{base_url}}/employees

Body:

```
{
  "name": "{{employee_name}}",
  "email": "{{employee_email}}",
  "department": "{{department}}",
  "salary": {{salary}}
}
```

Now you can change the input values without changing the request itself.

This is the beginning of professional Postman usage.

10. What are Postman variables?

A **variable** is a reusable value.

Instead of:

http://localhost:8080

use:

{{base_url}}

Then:

{{base_url}}/employees

If:

base_url = http://localhost:8080

Postman sends:

http://localhost:8080/employees

Variables are specifically designed to let you reuse values and change them according to your working environment. [Postman Docs](#)

11. Create an environment

Create:

Employee-DEV

Variables:

Variable	Value
base_url	<u>http://localhost:8080</u>
employee_id	101
token	abc123

Then request:

GET {{base_url}}/employees/{{employee_id}}

Postman sends:

GET http://localhost:8080/employees/101

12. DEV, TEST and PROD environments

This is one of the most important real-world uses.

Create:

DEV

base_url = http://localhost:8080

TEST

base_url = https://test-api.company.com

PROD

base_url = https://api.company.com

Your request stays:

GET {{base_url}}/employees

When you select DEV:

http://localhost:8080/employees

When you select TEST:

https://test-api.company.com/employees

When you select PROD:

https://api.company.com/employees

Postman environments are designed for exactly this kind of context-specific variable switching.
[Postman Docs](#)

13. Different inputs using variables

Create:

POST {{base_url}}/employees

Body:

```
{
  "name": "{{employee_name}}",
  "email": "{{employee_email}}",
  "department": "{{department}}",
  "salary": {{salary}}
}
```

Variables:

Variable	Value
employee_name	Anurag
employee_email	<u>anurag@example.com</u>
department	IT
salary	80000

Postman sends:

```
{
  "name": "Anurag",
  "email": "anurag@example.com",
  "department": "IT",
  "salary": 80000
}
```

Change the variables:

Variable	New Value
employee_name	Rahul
employee_email	<u>rahul@example.com</u>
department	HR
salary	70000

The same request now sends:

```
{
  "name": "Rahul",
  "email": "rahul@example.com",
  "department": "HR",
  "salary": 70000
}
```

```
"department": "HR",  
"salary": 70000  
}
```

14. PUT API

Suppose:

PUT /employees/101

Postman:

PUT
{{base_url}}/employees/{{employee_id}}

Body:

```
{  
  "name": "Anurag Bharti",  
  "email": "anurag.bharti@example.com",  
  "department": "Engineering",  
  "salary": 100000  
}
```

Here:

{{employee_id}}

is dynamically replaced.

15. PATCH API

PATCH is commonly used for partial updates.

PATCH {{base_url}}/employees/{{employee_id}}

Body:

```
{  
  "salary": 110000  
}
```

Instead of sending the entire employee object, you're requesting a partial change.

16. DELETE API

DELETE {{base_url}}/employees/{{employee_id}}

For example:

DELETE http://localhost:8080/employees/101

Possible response:

204 No Content

17. Headers

Headers provide additional information about the HTTP request.

Common headers:

Header	Example
Content-Type	application/json
Accept	application/json
Authorization	Bearer {{token}}
X-API-Key	{{api_key}}

For JSON:

Content-Type: application/json

Postman can automatically set appropriate content types for several body modes; for raw bodies, you can select the desired format or manually override the header. [Postman Docs](#)

18. Authentication

Suppose your Spring Boot API requires JWT authentication.

Login:

POST {{base_url}}/auth/login

Body:

```
{
  "username": "anurag",
  "password": "password"
}
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1Ni..."
}
```

Save the token into:

{{token}}

Then subsequent APIs use:

Authorization: Bearer {{token}}

For example:

GET {{base_url}}/employees

with:

Authorization
↓
Bearer Token
↓
{{token}}

19. API chaining

This is where Postman becomes much more powerful.

Imagine:

Login
↓
Token
↓
Create Employee
↓
Employee ID
↓
Get Employee
↓
Update Employee

↓
Delete Employee

Suppose Login returns:

```
{  
  "token": "ABC123"  
}
```

A post-response script can save it:

```
const response = pm.response.json();  
  
pm.environment.set("token", response.token);
```

Now another API can use:

Bearer {{token}}

20. Extract an ID from one API

Create Employee response:

```
{  
  "id": 501,  
  "name": "Anurag",  
  "department": "IT"  
}
```

Post-response script:

```
const response = pm.response.json();  
  
pm.environment.set("employee_id", response.id);
```

Now your next API can be:

GET {{base_url}}/employees/{{employee_id}}

which becomes:

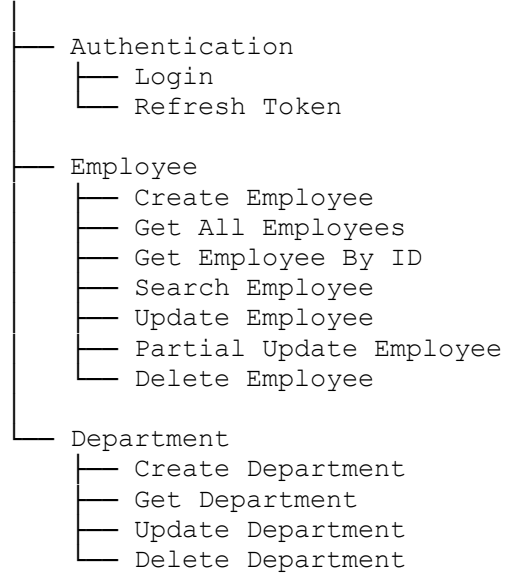
GET http://localhost:8080/employees/501

Postman supports passing data between requests using variables and scripts. [Postman Docs](#)

21. Create different APIs in one collection

A practical Spring Boot project could look like this:

Employee Management API



This is much better than keeping hundreds of unrelated requests.

22. Example: Search API with multiple parameters

Suppose your API is:

GET /employees

Parameters:

department
city
minSalary
maxSalary
page
size

Postman:

KEY	VALUE
department	IT
city	Mumbai

minSalary	50000
maxSalary	150000
page	0
size	10

Request:

GET {{base_url}}/employees

Actual URL:

GET
 {{base_url}}/employees?department=IT&city=Mumbai&minSalary=50000&maxSalary=150000&page=0&size=10

23. Same API with different parameters

You can test:

Test 1

department=IT
 city=Mumbai

Test 2

department=HR
 city=Delhi

Test 3

department=Finance
 city=Bangalore

Test 4

department=IT
 minSalary=100000

The request doesn't change:

GET {{base_url}}/employees

Only the parameter values change.

24. Different request bodies

Suppose:

POST {{base_url}}/employees

Input 1

```
{  
  "name": "Anurag",  
  "department": "IT",  
  "salary": 80000  
}
```

Input 2

```
{  
  "name": "Rahul",  
  "department": "HR",  
  "salary": 70000  
}
```

Input 3

```
{  
  "name": "Priya",  
  "department": "Finance",  
  "salary": 90000  
}
```

You can manually change the body and click **Send**.

But for dozens or hundreds of inputs, use **data-driven testing**.

25. Data-driven testing

Suppose you have:

```
employees.csv  
name,email,department,salary  
Anurag,anurag@example.com,IT,80000  
Rahul,rahul@example.com,HR,70000
```

Priya,priya@example.com,Finance,90000
Amit,amit@example.com,IT,85000

Your Postman request:

POST {{base_url}}/employees

Body:

```
{
  "name": "{{name}}",
  "email": "{{email}}",
  "department": "{{department}}",
  "salary": {{salary}}
}
```

Now one request can be executed repeatedly using different data.

Conceptually:

employees.csv

Row 1	→	POST	→	Anurag
Row 2	→	POST	→	Rahul
Row 3	→	POST	→	Priya
Row 4	→	POST	→	Amit

26. JSON test data

You can also use JSON data:

```
[
  {
    "name": "Anurag",
    "email": "anurag@example.com",
    "department": "IT",
    "salary": 80000
  },
  {
    "name": "Rahul",
    "email": "rahul@example.com",
    "department": "HR",
    "salary": 70000
  },
  {
    "name": "Priya",
```

```
"email": "priya@example.com",
"department": "Finance",
"salary": 90000
}
]
```

The same request can then be run for multiple iterations.

27. Collection Runner

The **Collection Runner** executes some or all requests in a collection in a chosen order. It can use scripts to pass data between requests and supports repeated iterations. [Postman Docs](#)

Suppose your collection is:

```
Employee API
├── Login
├── Create Employee
├── Get Employee
├── Update Employee
└── Delete Employee
```

Click:

```
Collection
↓
Run
↓
Select Environment
↓
Configure Iterations
↓
Start Run
```

Postman records the results of the requests during the collection run. [Postman Docs](#)

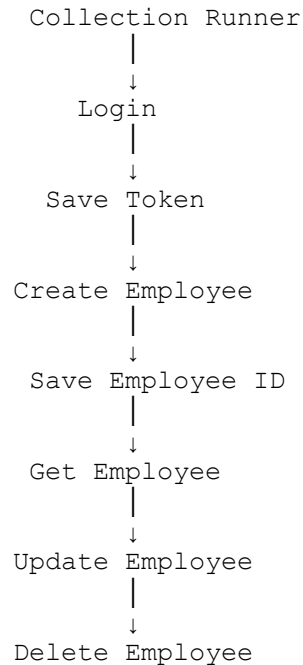
28. Run multiple APIs sequentially

Suppose:

1. Login
2. Create Employee
3. Get Employee

- 4. Update Employee
- 5. Delete Employee

Collection Runner:



This is a real-world API workflow.

29. Add API tests

Suppose the API should return HTTP 200.

In the request's **Scripts** → **Post-response** section:

```
pm.test("Status code is 200", function () {  
  pm.response.to.have.status(200);  
});
```

Postman runs the test after the response is received. The official quick start uses the same post-response scripting mechanism for basic API tests. [Postman Docs](#)

30. Test response JSON

Suppose:

```
{
  "id": 101,
  "name": "Anurag",
  "department": "IT"
}
```

Test:

```
const response = pm.response.json();

pm.test("Employee ID exists", function () {
  pm.expect(response.id).to.exist;
});

pm.test("Employee name is correct", function () {
  pm.expect(response.name).to.eql("Anurag");
});

pm.test("Department is IT", function () {
  pm.expect(response.department).to.eql("IT");
});
```

31. Test response time

```
pm.test("Response time is below 1000 ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(1000);
});
```

This gives you a basic response-time assertion.

32. Negative testing

Don't test only successful requests.

For:

POST /employees

test invalid inputs too.

Test 1 — Missing name

```
{
  "email": "test@example.com",
  "department": "IT"
}
```

Expected:

400 Bad Request

Test 2 — Invalid email

```
{  
  "name": "Anurag",  
  "email": "invalid-email",  
  "department": "IT"  
}
```

Expected:

400 Bad Request

Test 3 — Invalid employee ID

GET /employees/999999

Expected:

404 Not Found

Test 4 — No authentication

Expected:

401 Unauthorized

This is important because API testing isn't simply:

"Does the API return 200?"

It is:

"Does the API return the correct result for valid **and invalid** inputs?"

33. Complete example — Spring Boot Employee API

Assume:

Spring Boot
localhost:8080

Environment:

base_url = http://localhost:8080

Collection:

Employee API

API 1 — Login

POST {{base_url}}/auth/login

Body:

```
{  
  "username": "admin",  
  "password": "admin123"  
}
```

Response:

```
{  
  "token": "ABC123"  
}
```

Save:

token = ABC123

API 2 — Create employee

POST {{base_url}}/employees

Header:

Authorization: Bearer {{token}}

Body:

```
{  
  "name": "Anurag",  
  "email": "anurag@example.com",  
  "department": "IT",  
  "salary": 80000  
}
```

Response:

```
{
```

```
"id": 501,  
"name": "Anurag",  
"email": "anurag@example.com",  
"department": "IT",  
"salary": 80000  
}
```

Save:

employee_id = 501

API 3 — Get employee

GET {{base_url}}/employees/{{employee_id}}

Result:

GET http://localhost:8080/employees/501

API 4 — Update employee

PUT {{base_url}}/employees/{{employee_id}}

Body:

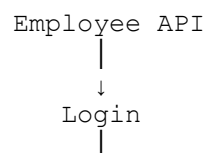
```
{  
  "name": "Anurag Bharti",  
  "email": "anurag.bharti@example.com",  
  "department": "Engineering",  
  "salary": 100000  
}
```

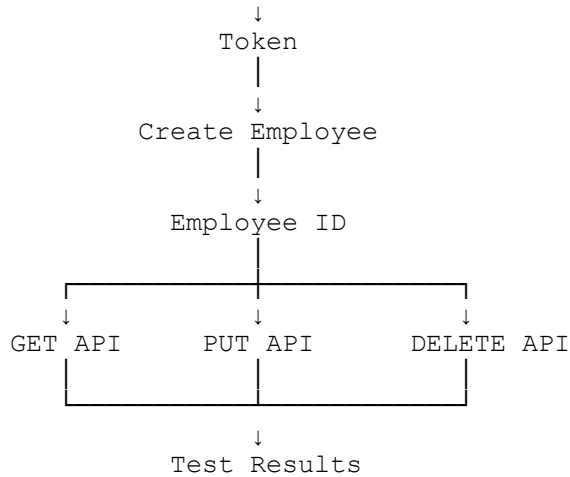
API 5 — Delete employee

DELETE {{base_url}}/employees/{{employee_id}}

34. Complete flow

Your entire Postman project becomes:





35. How to decide what to create in Postman

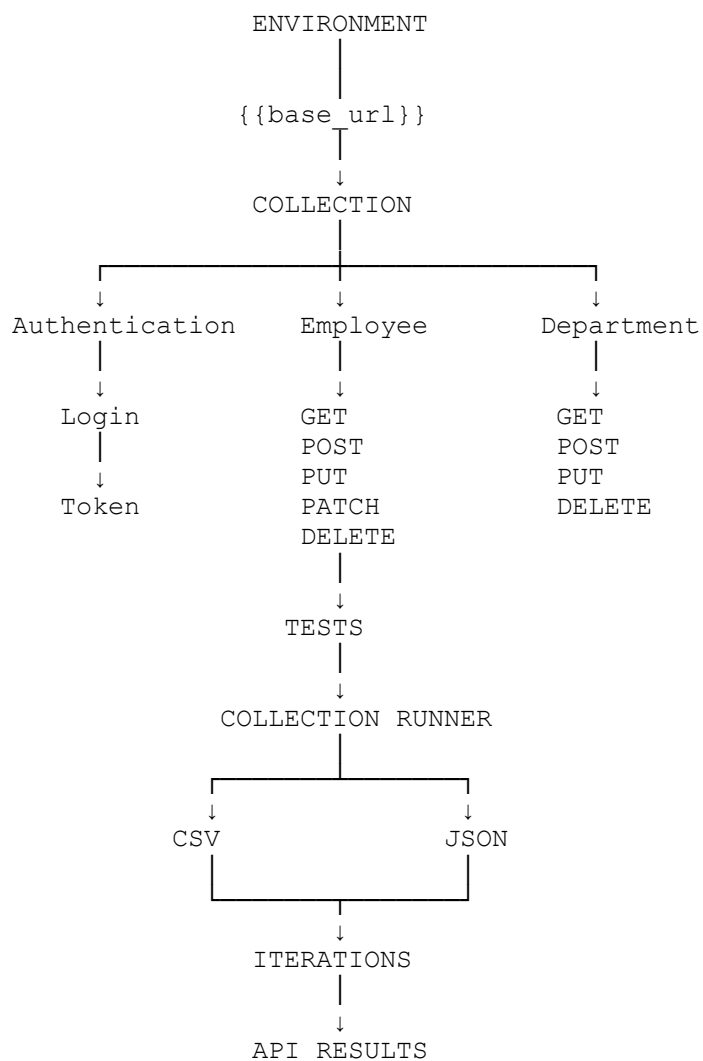
Use this rule:

Requirement	Postman feature
Retrieve data	GET request
Create data	POST request
Replace resource	PUT request
Partially update	PATCH request
Delete data	DELETE request
Resource ID	Path parameter
Filtering/search	Query parameter
JSON input	Raw → JSON
File upload	Form-data
Authentication	Authorization/Header
Reusable URL	Variable
DEV/TEST/PROD	Environments
Multiple related APIs	Collection

Pass token/ID between APIs	Variables + scripts
Test response	Post-response script
Same API with many inputs	CSV/JSON + Runner
Run multiple APIs	Collection Runner
CI/CD automation	Postman CLI

36. The most important architecture to remember

For professional Postman work, use this structure:



In short

The progression you should learn is:

1. Create Request → 2. Select HTTP Method → 3. Enter URL → 4. Add Path/Query Parameters → 5. Add Headers → 6. Add Authentication → 7. Add Body → 8. Send → 9. Validate Response → 10. Convert values to Variables → 11. Create Environment → 12. Save requests in Collection → 13. Chain APIs → 14. Add Tests → 15. Use CSV/JSON for different inputs → 16. Run with Collection Runner → 17. Automate.

Postman's current documentation explicitly supports this workflow: requests can contain parameters, body data and headers; collections organize reusable requests; environments manage context-specific values; and Collection Runner executes requests in sequence with scripts and iterations. [Postman Docs](#)